Clickpoint AI Chatbot — Production Developer Architecture Guide

RAG Intelligence, PostgreSQL pgvector, Automatic Admin Reindexing, and Resilient Dual-Mode Vercel Fallback

Production Grade | Next.js 14 App Router | Supabase PostgreSQL | Prisma ORM 7.9 | pgvector 384-dim | FastAPI Python Microservices | Better Auth | Cloudinary CDN | Version 2.0 (Oct 2026)

Clickpoint AI Chatbot — Production Developer Architecture & Operations Guide

> **Target Audience:** Core Developers, DevOps Engineers, and System Architects.
> **Status:** Production-Ready
> **Last Updated:** October 2026
> **Repository:** `clickpoint-innovation`

Table of Contents

1. Executive Summary & Core Objectives
2. High-Level System Architecture
   - Dual-Mode Deployment Model
   - Architectural Topology Diagram
3. Deep-Dive into Recent Production Changes
   - Automated CMS Mutation Reindexing Pipeline
   - Dynamic Data Grounding (Purge of Hardcoded Content)
   - Domain Candidate Injection & Semantic Re-Ranking
   - Strict Typography & Zero-Emoji Directives
   - Resilient Multi-Tier Chat API Route (`/api/chat`)
4. Database & Vector Store Schema Specifications
   - PostgreSQL `knowledge_chunks` Table
   - Dynamic Prisma Ingestion Mapping

5. Microservices & Endpoint Specifications
    - Ingestion Service (`apps/ai/ingestion-service`)
    - Chat Service (`apps/ai/chat-service`)
    - Next.js Web Route (`src/app/api/chat/route.ts`)
6. Environment Variables & Configuration Matrix
7. Developer Runbook: Operations, Testing & Maintenance
    - Local Development & Docker Orchestration
    - Manual Reindex Trigger
    - Cache Invalidation & Redis Maintenance
    - Verification & Automated Smoke Tests
    - Troubleshooting & Quota Management

---

1. Executive Summary & Core Objectives

The Clickpoint AI Chatbot (**Clix**) is an enterprise-grade, retrieval-augmented conversational agent designed to provide accurate, real-time information about Clickpoint Innovation's digital product engineering capabilities, team leadership, active career vacancies, portfolio case studies, technical blogs, pricing tiers, and client reviews.

Key Goals Delivered:

1. **100% Dynamic Synchronization:** Whenever an administrator creates, modifies, or deletes content anywhere on the CMS (e.g. posting a new vacancy or editing the About Us team list), the knowledge base vector embeddings and response caches update automatically without developer intervention.
2. **Zero Hardcoded Mock Data:** All company information, team members, pricing models, and service descriptions are sourced live from PostgreSQL.
3. **High-Resilience Dual-Mode Architecture:** The system runs at peak performance using a dedicated self-hosted Docker microservice stack (`pgvector` + FastAPI + Redis) on VPS/Docker, while seamlessly falling back to direct Prisma context extraction + Google Gemini when deployed serverless on Vercel.
4. **Clean Enterprise Presentation:** Responses are delivered in structured Markdown with strict adherence to a zero-emoji typographical policy and verified single-slash internal routing links.

---

2. High-Level System Architecture

Dual-Mode Deployment Model

The chatbot architecture balances two operational targets:

- **Target A: Self-Hosted Container Stack (VPS / Docker Desktop / Cloud VM):** Uses Python FastAPI microservices for vector indexing and retrieval. Embeddings are generated with `sentence-`

`transformers/all-MiniLM-L6-v2` (384-dimensional dense vectors) stored in PostgreSQL with the `pgvector` extension. Conversation state and generated responses are cached in Redis for sub-50ms repeat queries.

- **Target B: Serverless Edge/Node (Vercel Production Deployment):** When running on Vercel without direct internal network access to containerized microservices, `src/app/api/chat/route.ts` executes a direct Prisma batch extraction across 13 PostgreSQL tables, constructs a structured grounding context, and synthesizes answers using Google Gemini REST APIs with automatic multi-model fallback.

Architectural Topology Diagram

```
flowchart TD
    subgraph ClientLayer [Client & Web Presentation]
        User([Website Visitor])
        Admin([CMS Administrator])
        Widget[Floating Chatbot Widget Clix]
    end

    subgraph NextJSLayer [Next.js Application Layer - Port 3000 / Vercel]
        ChatRoute["POST /api/chat"]
        AdminActions["Server Actions & Admin API Routes"]
        PrismaORM["Prisma ORM Client"]
    end

    subgraph DockerLayer [Dedicated Python Microservices Stack]
        IngestionService["Ingestion Service (FastAPI: 8001)"]
        ChatService["Chat Service (FastAPI: 8002)"]
        RedisCache[("Redis Cache: 6379")]
    end

    subgraph DataLayer [Cloud Infrastructure - Supabase AWS ap-southeast-1]
        PostgresDB[("Supabase PostgreSQL")]
        PGVector[("pgvector: knowledge_chunks")]
    end

    subgraph ExternalLLM [External AI Infrastructure]
        GeminiAPI["Google Gemini API (flash-lite / 3.5 / 3.8)"]
    end

    %% User Interaction Flow
    User -->|Sends message| Widget
    Widget -->|JSON payload| ChatRoute

    %% Tier 1: Static Greeting (<10ms)
    ChatRoute -.->|Tier 1: Instant match| Widget

    %% Tier 2: Microservice Path
    ChatRoute -->|Tier 2: HTTP POST :8002| ChatService
    ChatService <-->|Check / Set Cache| RedisCache
    ChatService -->|Semantic Vector Search| PGVector
    ChatService -->|Prompt + Context| GeminiAPI

    %% Tier 3: Vercel Serverless Direct Fallback
    ChatRoute -->|Tier 3: Fallback if microservice unavailable| PrismaORM
    PrismaORM -->|Extract 13 tables| PostgresDB
    ChatRoute -->|Direct DB Context + Prompt| GeminiAPI

    %% Admin Reindexing Flow
    Admin -->|Updates CMS Content| AdminActions
    AdminActions -->|1. Mutate DB Records| PrismaORM
    AdminActions -->|2. Background HTTP POST /reindex| IngestionService
    IngestionService -->|Extract all DB sources| PostgresDB
    IngestionService -->|Generate 384-dim embeddings| PGVector
    IngestionService -->|Flush stale query cache| RedisCache
```

3. Deep-Dive into Recent Production Changes

Automated CMS Mutation Reindexing Pipeline

To eliminate stale or out-of-sync AI responses, background reindexing has been integrated directly into all admin mutation touchpoints using the helper utility:

```
// Located in: src/server/actions/* and src/app/api/*
function triggerReindexBackground() {
  const ingestionUrl = process.env.INGESTION_SERVICE_URL || "http://localhost:8001";
  const internalKey = process.env.INTERNAL_API_KEY || "clickpoint-secret-internal-key-2026";

  fetch(`${ingestionUrl}/reindex`, {
    method: "POST",
    headers: { "X-Internal-Key": internalKey },
    signal: AbortSignal.timeout(10000),
  }).catch(() => {});
}
```

Complete Inventory of Instrumented Mutations:

Domain	File Path	Operations Hooked
Services	`src/server/actions/services.ts`	`saveServicesPage`, `deleteService`
Blog	`src/server/actions/blog.ts`	`saveBlogPost`, `deleteBlogPost`
Case Studies	`src/server/actions/case-studies.ts`	`saveCaseStudy`, `deleteCaseStudy`
About Page	`src/server/actions/about.ts`	`saveAboutUs`, `deleteAboutUs`
Testimonials	`src/server/actions/testimonials.ts`	`approveTestimonial`, `deleteTestimonial`
FAQs	`src/app/api/faqs/route.ts`	`invalidateFaqCaches()` on mutations
FAQ Categories	`src/app/api/faq-categories/route.ts`	`invalidateFaqCaches()` on mutations
Job Vacancies	`src/app/api/careers/vacancies/route.ts`	`POST` (create), `PUT` (edit), `DELETE`
Careers Content	`src/app/api/careers/route.ts`	`PUT` (culture, perks, requirements)
Landing Page	`src/app/api/landing/route.ts`	`PUT` (hero, metrics, stats)
Chatbot Nodes	`src/app/api/chatbot/route.ts`	`PUT` (pricing tiers, node overrides)
Contact Page	`src/app/api/contact/route.ts`	`PUT` (emails, phone, physical address)
Journey Eras	`src/app/api/journey/route.ts`	`PUT` (milestones, eras, dates)
Industries	`src/app/api/industries/route.ts`	`PUT` (industry solutions, use cases)

Dynamic Data Grounding (Purge of Hardcoded Content)

In earlier iterations, team leadership and company statistics were partially populated with static fallback mocks. This has been completely purged:

- **Team & Leadership:** Sourced dynamically from `aboutPage.content.leadership.team`. Chunks are generated with the exact name, executive title, bio, and key competencies.
- **Vacancies:** Extracted directly from `jobVacancy` table where `status = 'ACTIVE'`.
- **Services & Methodologies:** Extracted directly from `servicesPage.content.categories` and `servicesPage.content.processSteps`.
- **Client Work & Proof:** Extracted from `caseStudy` (where `isPublished = true`) and `testimonial` (where `approved = true`).

Domain Candidate Injection & Semantic Re-Ranking

Dense vector semantic similarity can occasionally exhibit cosine drift for specialized keyword intents (for instance, asking *"What jobs or internships are open?"* might pull generic landing page chunks with high lexical similarity to *"Clickpoint"*).

To solve this, `apps/ai/chat-service/app/retrieval/retriever.py` incorporates **Domain Candidate Injection** and **Multi-Tier Dynamic Re-Ranking**:

```
# 1. Candidate Injection for Specific Domains
if has_career_q:
    cur.execute("""
        SELECT id, title, content, "sourceType", "sourceId", url, 0.45 as similarity
        FROM knowledge_chunks
        WHERE "sourceType" = 'career'
        LIMIT 15;
    """)
    rows.extend(cur.fetchall())

# 2. Dynamic Scoring & Re-ranking Formula
if has_career_q:
    if stype == "career":
        sim += 0.60
        if any(k in title_lower for k in ["opening", "vacancy", "developer", "marketting", "engir
            sim += 0.25
    else:
        sim -= 0.20  # Demote unrelated company fluff

# 3. Explicit Listing Intent Bypass
def detect_explicit_listing_intent(query: str) -> Optional[str]:
    # Instantly returns full table pool for explicit catalog/listing queries
```

Re-Ranking Rule Weights:

Intent / Domain	Target `sourceType`	Base Boost	Specific Keyword Modifier	Competitor Demotion
Leadership	`about`	`+0.40`	`+0.65` for `founder`, `ceo`, `cto`, `profile`	`-0.15` for mission/stats
Careers / Jobs	`career`	`+0.60`	`+0.25` for `opening`, `vacancy`, `intern`	`-0.20` for non-career
Blog / Tech	`blogPost`	`+0.50`	`+0.15` for author/date matches	`-0.15` for non-blog
Case Studies	`caseStudy`	`+0.50`	`+0.20` for client/result matches	`-0.15` for non-case
Services	`service`	`+0.35`	`+0.20` for engineering/capabilities	`-0.10` for non-service
Pricing	`service` / `faq`	`+0.35`	`+0.30` for package/rate/pricing chunks	`-0.15` for testimonials

---

Strict Typography & Zero-Emoji Directives

Per product requirements, the chatbot maintains a clean, professional, and typographic presentation:

1. **Purged Emojis:** The prompt, intent handler, and response sanitizers strictly prohibit waving hands (👋), rockets (🚀), brains, or other emoji icons.
2. **Regex Sanitization:** All outgoing LLM replies pass through bidirectional sanitizers before dispatch:
   - **Python (`apps/ai/chat-service/app/api/chat.py`):**

```
def clean_output_markdown(text: str) -> str:
    text = re.sub(r'\]\(\/{2,}', '](/', text)          # Fix double slashes
    text = re.sub(r'[\U00010000-\U0010ffff]', '', text)  # Strip emojis
    return text.strip()
```

   - **TypeScript (`src/app/api/chat/route.ts`):**

```
function sanitizeReply(text: string): string {
  return text
    .replace(/\]\(\/{2,}/g, "](")
    .replace(/[\u{1F300}-\u{1FAFF}\u{2600}-\u{27BF}]/gu, "")
    .trim();
}
```

3. **Canonical Link Enforcement:** Internal markdown links are sanitized to use a single leading slash (`[Careers](/careers)` instead of `[Careers](//careers)`).

Resilient Multi-Tier Chat API Route (`/api/chat`)

The client widget talks to a single unified Next.js route (`src/app/api/chat/route.ts`) that executes an ordered 4-tier resolution chain:

```
[Request: POST /api/chat]
          |
          ▼
[Tier 1: Static Greeting Check] ──────────── (Matches "hi", "hello", "who are you")
          | (No match)                                  |
          ▼                                             ▼
[Tier 2: Python FastAPI Microservice] ──> [Sub-10ms Structured Greeting]
          | (Timeout / Localhost on Vercel)
          ▼
[Tier 3: Live Direct Supabase + Gemini] ──> [Dynamic Answer from Live DB]
          | (Gemini network failure)
          ▼
[Tier 4: Offline Safety Fallback] ──────────> [Graceful Company Contact Info]
```

---

4. Database & Vector Store Schema Specifications

PostgreSQL `knowledge_chunks` Table

Created in the primary Supabase PostgreSQL database under schema `public` :

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS knowledge_chunks (
    id VARCHAR(255) PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    content TEXT NOT NULL,
    embedding vector(384),
    "sourceType" VARCHAR(50) NOT NULL,
    "sourceId" VARCHAR(255) NOT NULL,
    url VARCHAR(255),
    "updatedAt" TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_chunks_source ON knowledge_chunks("sourceType");
CREATE INDEX IF NOT EXISTS idx_chunks_source_id ON knowledge_chunks("sourceId");
CREATE INDEX IF NOT EXISTS idx_chunks_embedding ON knowledge_chunks USING ivfflat (embedding vect
```

Dynamic Prisma Ingestion Mapping

The ingestion service reads from these 13 Prisma tables and translates records into standard vector chunks:

Source Type	Prisma Model Source	Chunk ID Format	Title & URL Mapping
`about`	`prisma.aboutPage`	`about-leadership-{id}-{index}`	`Leadership Profile: {name} ({role})` -> `/about#team`
`about`	`prisma.aboutPage`	`about-mission-0`	`Clickpoint Mission & Company Story` -> `/about#mission`
`service`	`prisma.servicesPage`	`service-{id}-{index}`	`Service Offering: {name}` -> `/services#{slug}`
`career`	`prisma.jobVacancy`	`career-{id}-0`	`Career Opening: {title} ({location})` -> `/careers`
`career`	`prisma.careersPage`	`career-culture-{index}`	`Careers Culture, Benefits & Engineering Perks` -> `/careers#culture`
`caseStudy`	`prisma.caseStudy`	`case-study-{id}-0`	`Case Study: {title}` -> `/case-studies/{slug}`
`blogPost`	`prisma.blogPost`	`blog-{id}-0`	`Blog Article: {title}` -> `/blog/{slug}`
`testimonial`	`prisma.testimonial`	`testimonial-{id}-0`	`Testimonial from {clientName} ({company})` -> `/testimonials`
`faq`	`prisma.fAQ` / `fAQCategory`	`faq-{id}-0`	`FAQ: {question}` -> `/faqs`
`journey`	`prisma.journeyPage`	`journey-era-{index}`	`Clickpoint Journey Era ({year})` -> `/journey`
`industry`	`prisma.industriesPage`	`industry-{id}-{index}`	`Industry Solution: {title}` -> `/industries`
`contact`	`prisma.contactPage`	`contact-details-0`	`Official Company Contact & Location` -> `/contact`
`landing`	`prisma.landingPage`	`landing-metrics-0`	`Company Metrics & Achievements` -> `/#stats`

5. Microservices & Endpoint Specifications

Ingestion Service (`apps/ai/ingestion-service`)

- **Base Port:** `8001`
- **Model:** `sentence-transformers/all-MiniLM-L6-v2`
- **Internal Authentication Header:** `X-Internal-Key: <INTERNAL_API_KEY>`

Endpoints:

- `POST /reindex`
  - Re-reads all 13 Prisma tables from PostgreSQL.
  - Chunks, tokens, and normalizes texts.
  - Generates 384-dimensional vector embeddings.
  - Atomically truncates and re-populates `knowledge_chunks`.
  - Sends a `FLUSHALL` signal to Redis to purge stale response caches.
  - **Response:** `{"status": "success", "indexed_chunks": 125}`
- `POST /index-chunk`
  - Incremental single-chunk indexation.
  - **Body:** `{ "id": "...", "title": "...", "content": "...", "sourceType": "...", "sourceId": "...", "url": "..." }`
- `DELETE /chunks/{sourceId}`
  - Deletes all chunks associated with a specific entity (e.g. on blog post deletion).

Chat Service (`apps/ai/chat-service`)

- **Base Port:** `8002`
- **Internal Authentication Header:** `X-Internal-Key: <INTERNAL_API_KEY>`

Endpoints:

- `POST /chat` & `POST /api/chat`
  - **Request Body:**

```
{
  "message": "Who is Rabin Shrestha and what is his role?",
  "sessionId": "optional-uuid"
}
```

  - **Processing Flow:**
    1. Checks static greeting and intent.
    2. Hashes query and checks Redis cache key (`chat_cache:<md5>`).
    3. Runs candidate injection and `pgvector` similarity query.
    4. Applies domain re-ranking and isolates the top 5 chunks.
    5. Dispatches RAG context prompt to Google Gemini.
    6. Sanitizes output typography and links.
    7. Stores user and assistant messages in session history.
    8. Sets Redis cache with 24h TTL.

- **Response Body:**


```
{
  "reply": "Rabin Shrestha is the Chief Technology Officer (CTO & AI Architect) at Clickpo
  "sessionId": "80152c2a-8b25-44b4-b2c1-b3e6e6cdd18a",
  "sources": [
    {
      "title": "Leadership Profile: Rabin Shrestha (Chief Technology Officer (CTO & AI Arc
      "url": "/about#team",
      "sourceType": "about"
    }
  ],
  "cached": false,
  "provider_used": "gemini"
}
```

Next.js Web Route (`src/app/api/chat/route.ts`)

- **URL:** `POST /api/chat`
- **Public Endpoint:** Consumed by client widget component (`src/components/chatbot/chat-widget.tsx`).
- **Input Schema:** Accepts either `{"message": "..."}` or `{"prompt": "..."}` .
- **Output Schema:** Returns `{ reply: string, sources: Array<{ title, url, sourceType }>, sessionId: string }` .

---

6. Environment Variables & Configuration Matrix

Variable	Environment	Required In	Description
`DATABASE_URL`	Supabase Pooled URL	Next.js, FastAPI, Ingestion	`postgresql://postgres.[id]:[pass]@aws-0-ap-southeast-1.pooler.supabase.com:6543/postgres?pgbouncer=true`
`DIRECT_URL`	Supabase Direct URL	Next.js, Prisma, Ingestion	`postgresql://postgres:[pass]@db.[id].supabase.co:5432/postgres`
`REDIS_URL`	Redis Connection	Chat Service	Local: `redis://clickpoint-redis:6379/0` (or `redis://localhost:6380/0`)
`GEMINI_API_KEY`	Google AI Studio Key	Chat Service, Next.js	Used for Gemini LLM generation
`GEMINI_MODEL_NAME`	Model Identifier	Chat Service, Next.js	Default: `gemini-flash-lite-latest` or `gemini-3.5-flash-lite`
`INTERNAL_API_KEY`	Shared Secret	All Services	Internal auth token for service-to-service calls
`CHAT_SERVICE_URL`	URL to Chat Service	Next.js	Local: `http://localhost:8002`. On Vercel: left blank or set to VPS endpoint.
`INGESTION_SERVICE_URL`	URL to Ingestion	Next.js	Local: `http://localhost:8001`. On Vercel: left blank or set to VPS endpoint.
`VERCEL`	System Flag	Vercel Platform	Automatically set to `1` by Vercel; triggers Tier 3 direct DB RAG.

7. Developer Runbook: Operations, Testing & Maintenance

Local Development & Docker Orchestration

To run the entire ecosystem locally with Docker:

```
# 1. Start all container services
docker compose -f docker-compose.prod.yml up -d

# 2. Check running services
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Expected healthy containers:

- `clickpoint-postgres` (Port 5433 -> 5432)

- `clickpoint-redis` (Port 6380 -> 6379)
- `clickpoint-ingestion-service` (Port 8001 -> 8001)
- `clickpoint-chat-service` (Port 8002 -> 8002)
- `clickpoint-web` (Port 3001 -> 3000)

Manual Reindex Trigger

To manually trigger a complete database reindexing and vector rebuild:

```
curl -X POST http://localhost:8001/reindex \
  -H "X-Internal-Key: clickpoint-secret-internal-key-2026"
```

Expected Output:

```
{"status":"success","message":"Successfully reindexed knowledge base with 125 vector chunks.","ir
```

Cache Invalidation & Redis Maintenance

To flush all cached query responses:

```
# Via Docker
docker exec clickpoint-redis redis-cli flushall

# Or via Python Chat Service internal container
docker exec clickpoint-chat-service python -c "
import redis
r = redis.from_url('redis://redis:6379/0')
r.flushall()
print('Redis flushed successfully')
"
```

Verification & Automated Smoke Tests

Run these test queries to verify system health across both microservice and web layers:

```
# 1. Test Static Greeting (Sub-10ms response, zero emojis)
curl -s -X POST http://localhost:3001/api/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "Hello"}'

# 2. Test Dynamic Leadership Profile Retrieval
curl -s -X POST http://localhost:3001/api/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "Who is Rabin Shrestha and what is his role?"}'

# 3. Test Active Job Vacancies Retrieval
curl -s -X POST http://localhost:3001/api/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "What jobs or internships are currently open at Clickpoint Innovation?"}'

# 4. Test Engineering Services Retrieval
curl -s -X POST http://localhost:3001/api/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "What engineering services do you provide?"}'
```

Troubleshooting & Quota Management

1. **Google Gemini HTTP 429 Quota Exceeded:**
   - **Symptom:** Logs show `HTTP 429 ResourceExhausted` when requesting `gemini-3.7-flash`.
   - **Fix:** Both `apps/ai/chat-service/app/llm/provider.py` and `src/app/api/chat/route.ts` implement an automated fallback chain: `gemini-flash-lite-latest` -> `gemini-3.5-flash-lite` -> `gemini-3.1-flash-lite` -> `gemini-3.5-flash`.
   - **Action:** Ensure `GEMINI_MODEL_NAME=gemini-flash-lite-latest` is set in your environment for high-throughput, low-latency synthesis.
2. **Supabase Connection Pooling Timeouts (`P2024`):**
   - **Symptom:** Prisma reports connection timeout during concurrent reindexing.
   - **Fix:** Verify that `DATABASE_URL` connects through Supabase's transaction pooler (port `6543` with `?pgbouncer=true&connection_limit=15`) rather than direct port `5432`.
3. **Stale Query Response:**
   - **Symptom:** Modifying an admin item does not immediately reflect in the chat reply.
   - **Fix:** Flush Redis using `docker exec clickpoint-redis redis-cli flushall` and confirm that `triggerReindexBackground()` was invoked during the mutation.